

```
In [1]: print('Hello')
```

Hello

```
In [2]: print('World')
```

World

```
In [3]: import pycaret.datasets as pcd
data = pcd.get_data('diabetes')
```

	Number of times pregnant	Plasma glucose concentration a 2 hours in an oral glucose tolerance test	Diastolic blood pressure (mm Hg)	Triceps skin fold thickness (mm)	2- Hour serum insulin (mu U/ml)	Body mass index (weight in kg/ (height in m)^2)	Diabetes pedigree function	Age (years)	Class variable
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

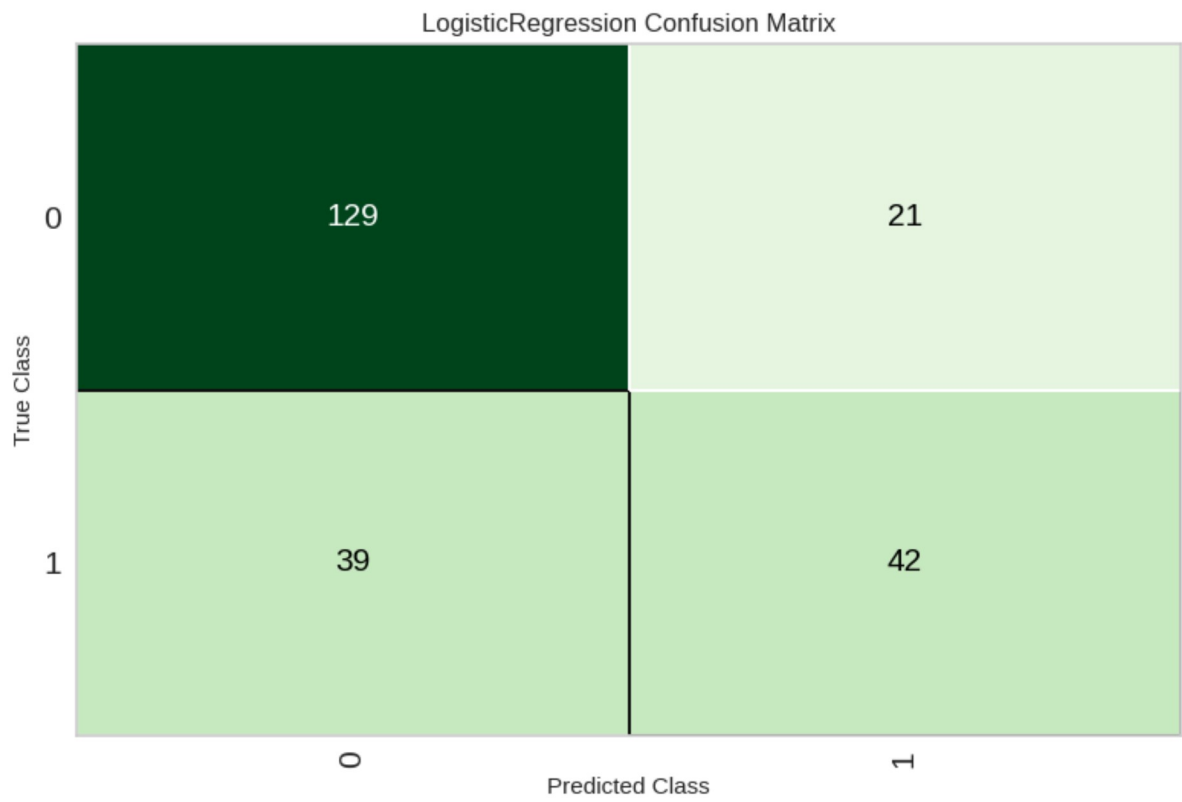
```
In [4]: import pycaret.classification as pcc
s = pcc.setup(data, target = 'Class variable', session_id = 42)
```

	Description	Value
0	Session id	42
1	Target	Class variable
2	Target type	Binary
3	Original data shape	(768, 9)
4	Transformed data shape	(768, 9)
5	Transformed train set shape	(537, 9)
6	Transformed test set shape	(231, 9)
7	Numeric features	8
8	Preprocess	True
9	Imputation type	simple
10	Numeric imputation	mean
11	Categorical imputation	mode
12	Fold Generator	StratifiedKFold
13	Fold Number	10
14	CPU Jobs	-1
15	Use GPU	False
16	Log Experiment	False
17	Experiment Name	clf-default-name
18	USI	f79e

In [5]: `best = pcc.compare_models()`

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)
lr	Logistic Regression	0.7784	0.8284	0.5819	0.7369	0.6397	0.4851	0.4994	0.6590
lda	Linear Discriminant Analysis	0.7784	0.8304	0.5819	0.7327	0.6395	0.4848	0.4977	0.0300
ridge	Ridge Classifier	0.7747	0.0000	0.5661	0.7344	0.6297	0.4740	0.4883	0.0240
nb	Naive Bayes	0.7599	0.8160	0.5930	0.6858	0.6275	0.4537	0.4616	0.0270
qda	Quadratic Discriminant Analysis	0.7525	0.8166	0.5775	0.6718	0.6149	0.4356	0.4424	0.0290
et	Extra Trees Classifier	0.7524	0.8100	0.5605	0.6824	0.5995	0.4275	0.4427	0.2230
gbc	Gradient Boosting Classifier	0.7449	0.8157	0.5398	0.6773	0.5874	0.4096	0.4239	0.1210
rf	Random Forest Classifier	0.7393	0.8114	0.5401	0.6601	0.5857	0.4005	0.4101	0.2340
ada	Ada Boost Classifier	0.7336	0.7854	0.5398	0.6396	0.5749	0.3865	0.3961	0.1080
lightgbm	Light Gradient Boosting Machine	0.7225	0.7811	0.5506	0.6231	0.5779	0.3739	0.3804	0.1220
knn	K Neighbors Classifier	0.7116	0.7582	0.5205	0.6137	0.5531	0.3448	0.3539	0.0370
dt	Decision Tree Classifier	0.6944	0.6529	0.5173	0.5689	0.5379	0.3118	0.3144	0.0290
dummy	Dummy Classifier	0.6518	0.5000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0570
svm	SVM - Linear Kernel	0.5694	0.0000	0.6307	0.4608	0.4685	0.1562	0.1922	0.0290

```
In [6]: pcc.plot_model(best, plot='confusion_matrix')
```



```
In [7]: print(pcc.convert_model(best))
```

```

-----
ModuleNotFoundError                                Traceback (most recent call last)
Cell In[7], line 1
----> 1 print(pcc.convert_model(best))

File /opt/conda/envs/pycaret/lib/python3.8/site-packages/pycaret/utils/generic.py:9
65, in check_if_global_is_not_none.<locals>.decorator.<locals>.wrapper(*args, **kwa
rgs)
    963     if globals_d[name] is None:
    964         raise ValueError(message)
--> 965 return func(*args, **kwargs)

File /opt/conda/envs/pycaret/lib/python3.8/site-packages/pycaret/classification/fun
ctional.py:3100, in convert_model(estimator, language)
    3050 @check_if_global_is_not_none(globals(), _CURRENT_EXPERIMENT_DECORATOR_DICT)
    3051 def convert_model(estimator, language: str = "python") -> str:
    3053     """
    3054     This function transpiles trained machine learning models into native
    3055     inference script in different programming languages (Python, C, Java,
    (...)
    3098
    3099     """
-> 3100     return _CURRENT_EXPERIMENT.convert_model(estimator, language)

File /opt/conda/envs/pycaret/lib/python3.8/site-packages/pycaret/internal/pycaret_e
xperiment/tabular_experiment.py:2527, in _TabularExperiment.convert_model(estimato
r, language)
    2477 @staticmethod
    2478 def convert_model(estimator, language: str = "python") -> str:
    2479     """
    2480     This function transpiles trained machine learning models into native
    2481     inference script in different programming languages (Python, C, Java,
    (...)
    2524
    2525     """
-> 2527     _check_soft_dependencies("m2cgen", extra=None, severity="error")
    2528     import m2cgen as m2c
    2530     if language == "python":

File /opt/conda/envs/pycaret/lib/python3.8/site-packages/pycaret/utils/_dependencie
s.py:152, in _check_soft_dependencies(package, severity, extra, install_name)
    150 if severity == "error":
    151     logger.exception(f"{msg}")
--> 152     raise ModuleNotFoundError(msg)
    153 elif severity == "warning":
    154     logger.warning(f"{msg}")

ModuleNotFoundError:
'm2cgen' is a soft dependency and not included in the pycaret installation. Please
run: `pip install m2cgen` to install.

```

```
In [8]: %pip install m2cgen
```

Collecting m2cgen

Downloading m2cgen-0.10.0-py3-none-any.whl (92 kB)

92.2/92.2 kB 3.0 MB/s eta 0:00:00

Requirement already satisfied: numpy in /opt/conda/envs/pycaret/lib/python3.8/site-packages (from m2cgen) (1.23.5)

Installing collected packages: m2cgen

Successfully installed m2cgen-0.10.0

Note: you may need to restart the kernel to use updated packages.

```
In [1]: import pycaret.datasets as pcd
data = pcd.get_data('diabetes')

import pycaret.classification as pcc
s = pcc.setup(data, target = 'Class variable', session_id = 42)

best = pcc.compare_models()

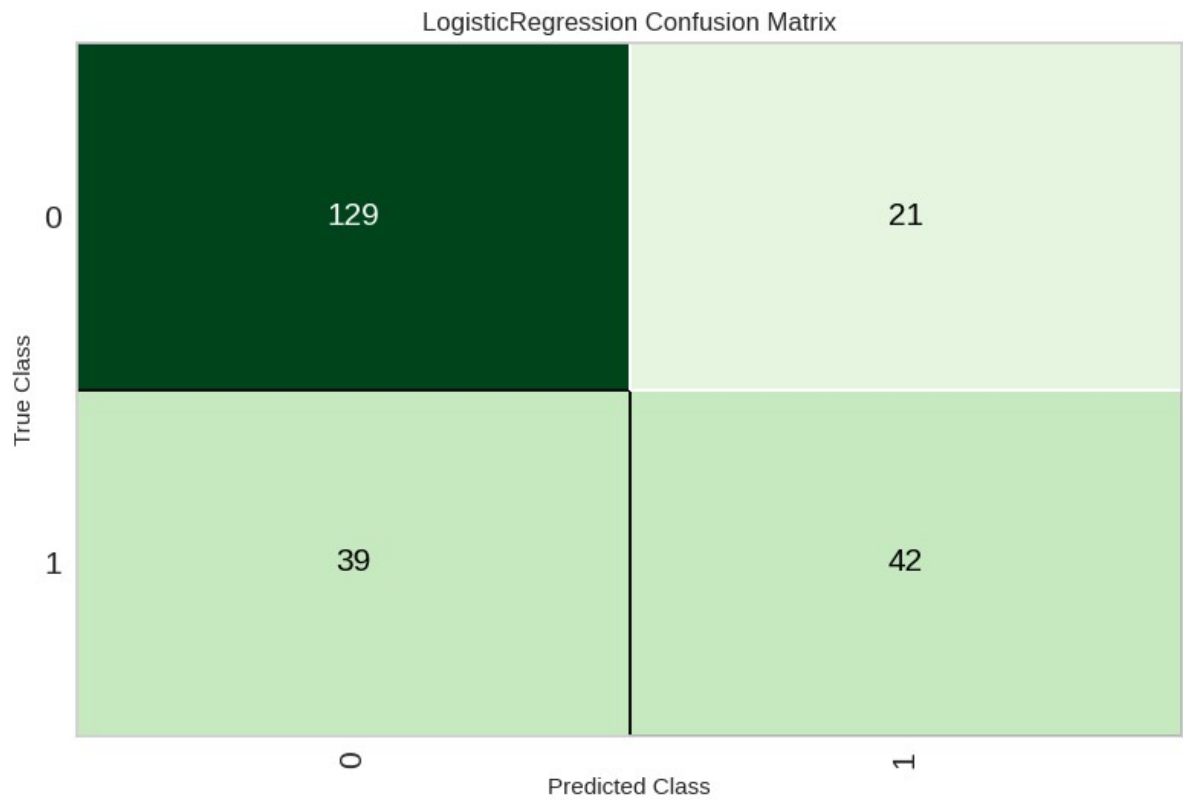
pcc.plot_model(best, plot='confusion_matrix')

print(pcc.convert_model(best))
```

	Number of times pregnant	Plasma glucose concentration a 2 hours in an oral glucose tolerance test	Diastolic blood pressure (mm Hg)	Triceps skin fold thickness (mm)	2- Hour serum insulin (mu U/ml)	Body mass index (weight in kg/ (height in m)^2)	Diabetes pedigree function	Age (years)	Class variable
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

	Description	Value
0	Session id	42
1	Target	Class variable
2	Target type	
3	Original data shape	
4	Transformed data shape	(768, 9)
5	Transformed train set shape	(537, 9)
6	Transformed test set shape	(231, 9)
7	Numeric features	8
8	Preprocess	True
9	Imputation type	simple
10	Numeric imputation	mean
11	Categorical imputation	mode
12	Fold Generator	StratifiedKFold
13	Fold Number	10
14	CPU Jobs	-1
15	Use GPU	False
16	Log Experiment	False
17	Experiment Name	clf-default-name
18	USI	e900

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)
lr	Logistic Regression	0.7784	0.8284	0.5819	0.7369	0.6397	0.4851	0.4994	0.2750
lda	Linear Discriminant Analysis	0.7784	0.8304	0.5819	0.7327	0.6395	0.4848	0.4977	0.0210
ridge	Ridge Classifier	0.7747	0.0000	0.5661	0.7344	0.6297	0.4740	0.4883	0.0170
nb	Naive Bayes	0.7599	0.8160	0.5930	0.6858	0.6275	0.4537	0.4616	0.0200
qda	Quadratic Discriminant Analysis	0.7525	0.8166	0.5775	0.6718	0.6149	0.4356	0.4424	0.0190
et	Extra Trees Classifier	0.7524	0.8100	0.5605	0.6824	0.5995	0.4275	0.4427	0.0690
gbc	Gradient Boosting Classifier	0.7449	0.8157	0.5398	0.6773	0.5874	0.4096	0.4239	0.0390
rf	Random Forest Classifier	0.7393	0.8114	0.5401	0.6601	0.5857	0.4005	0.4101	0.0710
ada	Ada Boost Classifier	0.7336	0.7854	0.5398	0.6396	0.5749	0.3865	0.3961	0.0490
lightgbm	Light Gradient Boosting Machine	0.7225	0.7811	0.5506	0.6231	0.5779	0.3739	0.3804	0.0250
knn	K Neighbors Classifier	0.7116	0.7582	0.5205	0.6137	0.5531	0.3448	0.3539	0.0260
dt	Decision Tree Classifier	0.6944	0.6529	0.5173	0.5689	0.5379	0.3118	0.3144	0.0200
dummy	Dummy Classifier	0.6518	0.5000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0190
svm	SVM - Linear Kernel	0.5694	0.0000	0.6307	0.4608	0.4685	0.1562	0.1922	0.0180



```
def score(input):  
    return -8.54519153114238 + input[0] * 0.13864844052315797 + input[1] * 0.036624  
269755191174 + input[2] * -0.011052761659724268 + input[3] * -0.002393349968874761  
+ input[4] * -0.0007502209348825264 + input[5] * 0.09341933236931786 + input[6] * 0  
.6331540967206996 + input[7] * 0.007899638208096879
```

```

In [17]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import (
    accuracy_score, roc_auc_score, recall_score,
    precision_score, f1_score, cohen_kappa_score, matthews_corrcoef
)

# 1. Extract test features and true target labels from the active experiment
X_test = pcc.get_config('X_test')
y_test = pcc.get_config('y_test')

# 2. Predict raw probability arrays for Logistic Regression (class 1)
# (Adjust 'best_lr_model' to match your actual variable name)
probabilities = best.predict_proba(X_test)[:, 1]

# 3. Compute the unchanging AUC score for reference
auc_score = roc_auc_score(y_test, probabilities)

# 4. Cycle through thresholds from 0.0 up to 1.0 to compute cutoff metrics
thresholds = np.linspace(0, 1, 100)
metrics_data = []

for t in thresholds:
    preds = (probabilities >= t).astype(int)
    metrics_data.append({
        'Threshold': t,
        'Accuracy': accuracy_score(y_test, preds),
        'Recall': recall_score(y_test, preds, zero_division=0),
        'Precision': precision_score(y_test, preds, zero_division=0),
        'F1-Score': f1_score(y_test, preds, zero_division=0),
        'Kappa': cohen_kappa_score(y_test, preds),
        'MCC': matthews_corrcoef(y_test, preds)
    })

df_metrics = pd.DataFrame(metrics_data)

# 5. Build and render the multi-metric optimization graph
plt.figure(figsize=(12, 7))

# Plot the threshold-dependent curves
plt.plot(df_metrics['Threshold'], df_metrics['Accuracy'], label='Accuracy', color='red')
plt.plot(df_metrics['Threshold'], df_metrics['Precision'], label='Precision', color='blue')
plt.plot(df_metrics['Threshold'], df_metrics['Recall'], label='Recall', color='red')
plt.plot(df_metrics['Threshold'], df_metrics['F1-Score'], label='F1-Score', color='blue')
plt.plot(df_metrics['Threshold'], df_metrics['Kappa'], label='Kappa (Cohen)', color='blue')
plt.plot(df_metrics['Threshold'], df_metrics['MCC'], label='MCC', color='red')

# Draw AUC as a static horizontal reference line
plt.axhline(y=auc_score, color='red', linestyle='--', alpha=0.5, label=f'AUC Baseline')

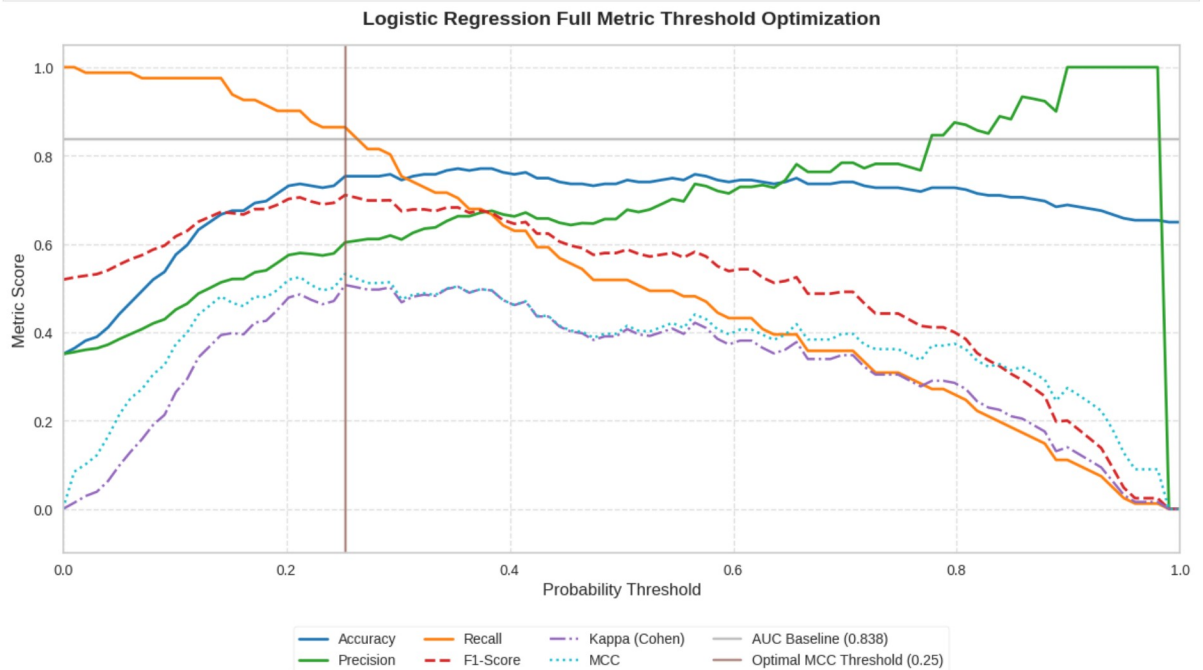
# Find and pinpoint the optimal threshold (using MCC as an example of a balanced metric)
best_mcc_idx = df_metrics['MCC'].idxmax()
best_thresh = df_metrics.loc[best_mcc_idx, 'Threshold']
plt.axvline(x=best_thresh, color='red', linestyle='--', alpha=0.7,
            label=f'Optimal MCC Threshold (best thresh: {best_thresh:.2f})')

```

```

# Graph aesthetics
plt.title('Logistic Regression Full Metric Threshold Optimization', fontsize=14, fo
plt.xlabel('Probability Threshold', fontsize=12)
plt.ylabel('Metric Score', fontsize=12)
plt.xlim(0, 1)
plt.ylim(-0.1, 1.05) # Lower bound accommodates negative MCC/Kappa if they occur
plt.legend(loc='lower center', bbox_to_anchor=(0.5, -0.25), ncol=4, frameon=True)
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()

```

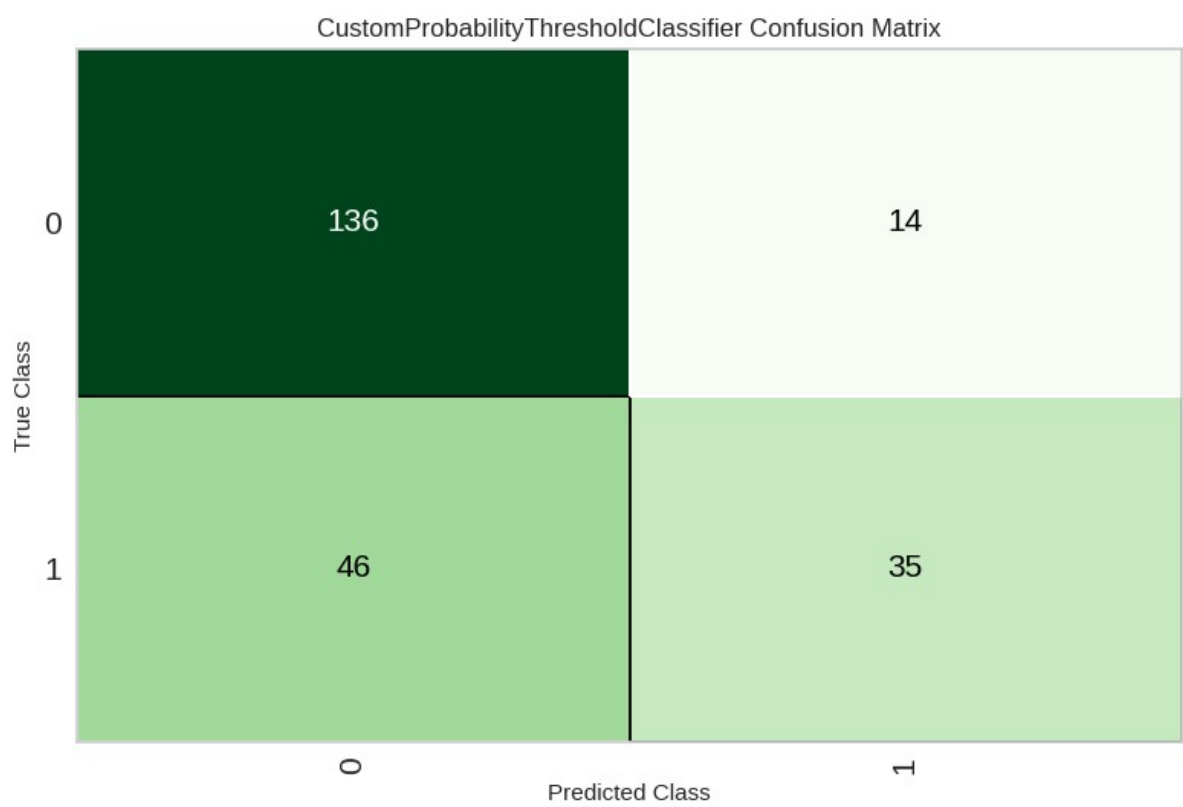


```

In [18]: opt = pcc.optimize_threshold(best, optimize='Accuracy')
pcc.plot_model(opt, plot = 'confusion_matrix')

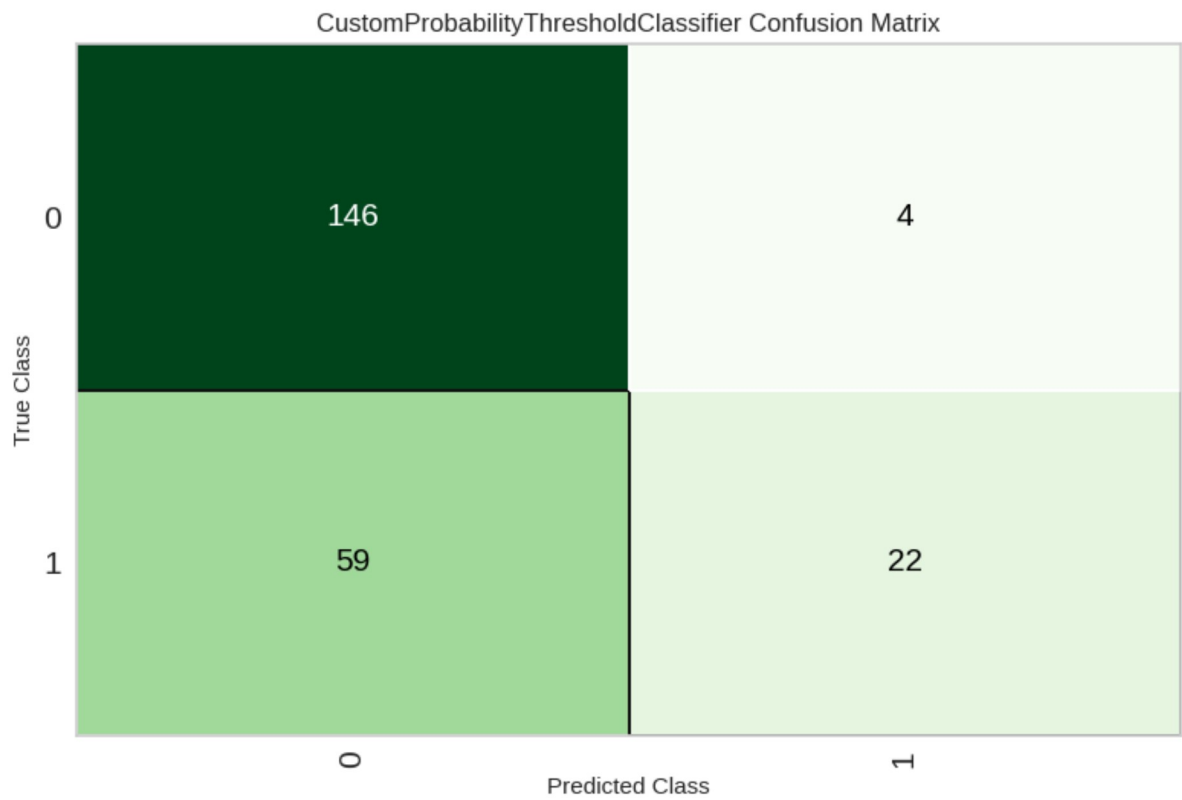
```

optimization loop finished successfully. Best threshold: 0.6015625 with Accuracy=0.784



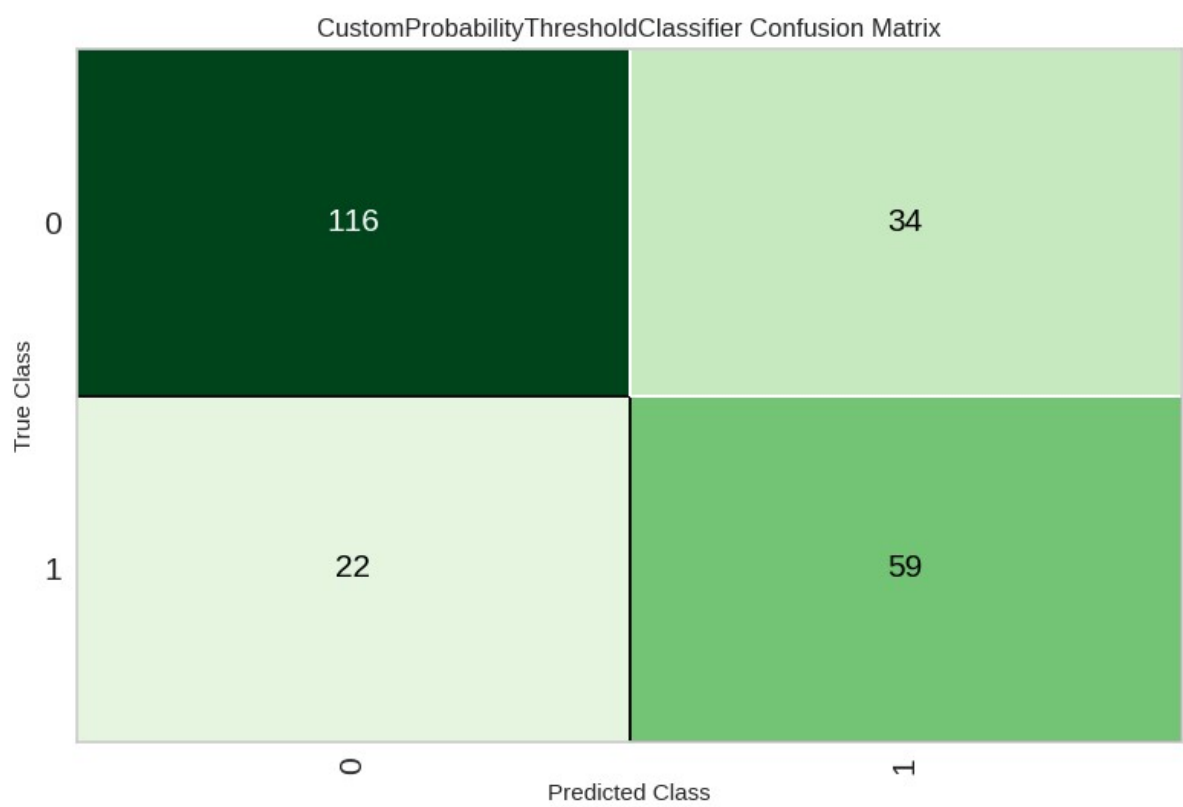
```
In [19]: optPrecision = pcc.optimize_threshold(best, optimize='Precision')
pcc.plot_model(optPrecision, plot = 'confusion_matrix')
```

optimization loop finished successfully. Best threshold: 0.7890625 with Prec.=0.9175



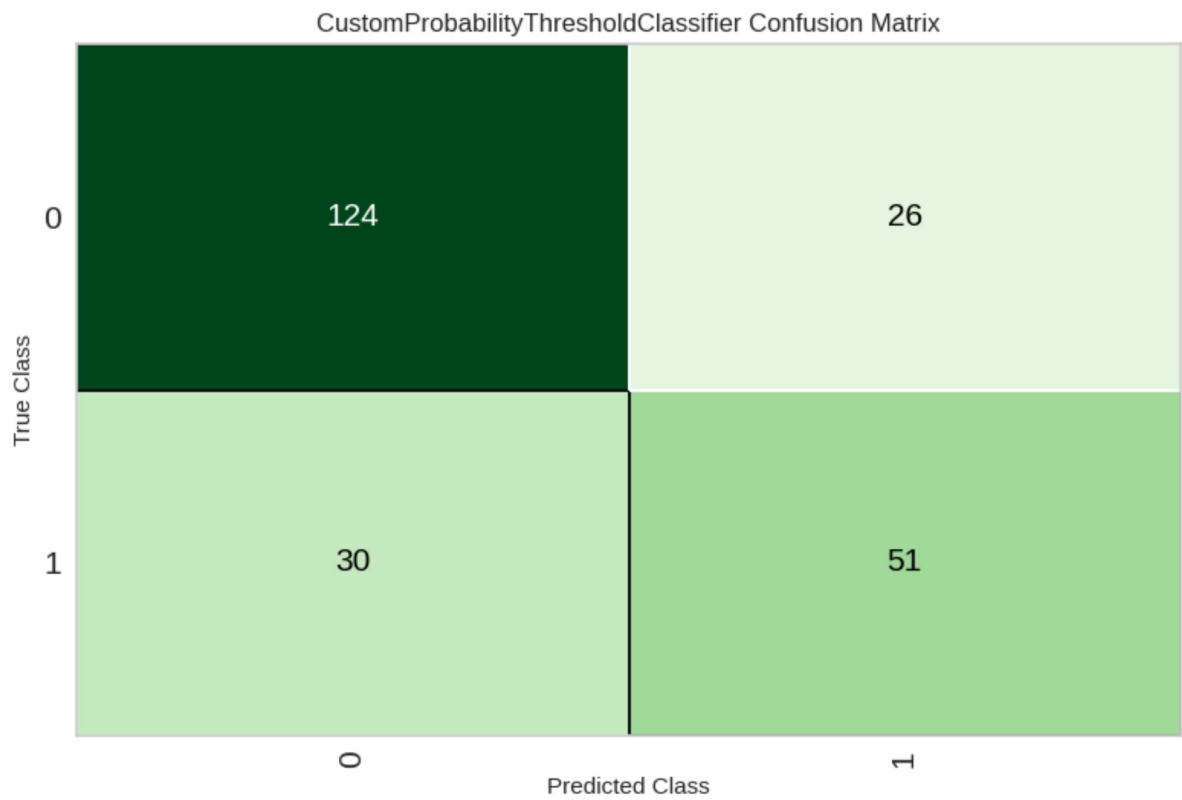
```
In [20]: optF1 = pcc.optimize_threshold(best, optimize = 'F1')  
pcc.plot_model(optF1, plot = 'confusion_matrix')
```

optimization loop finished successfully. Best threshold: 0.328125 with F1=0.6725



```
In [21]: optKappa = pcc.optimize_threshold(best, optimize = 'Kappa')
pcc.plot_model(optKappa, plot = 'confusion_matrix')

optimization loop finished successfully. Best threshold: 0.40625 with Kappa=0.4944
```

```
In [23]: best3 = pcc.compare_models(n_select=3, sort='F1')
stack = pcc.stack_models(best3, optimize='F1')
opstackF1 = pcc.optimize_threshold(stack, optimize='F1')
pcc.plot_model(opstackF1, 'confusion_matrix')
```

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)
lr	Logistic Regression	0.7784	0.8284	0.5819	0.7369	0.6397	0.4851	0.4994	0.1650
lda	Linear Discriminant Analysis	0.7784	0.8304	0.5819	0.7327	0.6395	0.4848	0.4977	0.1580
ridge	Ridge Classifier	0.7747	0.0000	0.5661	0.7344	0.6297	0.4740	0.4883	0.1590
nb	Naive Bayes	0.7599	0.8160	0.5930	0.6858	0.6275	0.4537	0.4616	0.1550
qda	Quadratic Discriminant Analysis	0.7525	0.8166	0.5775	0.6718	0.6149	0.4356	0.4424	0.1530
et	Extra Trees Classifier	0.7524	0.8100	0.5605	0.6824	0.5995	0.4275	0.4427	0.1960
gbc	Gradient Boosting Classifier	0.7449	0.8157	0.5398	0.6773	0.5874	0.4096	0.4239	0.1740
rf	Random Forest Classifier	0.7393	0.8114	0.5401	0.6601	0.5857	0.4005	0.4101	0.1900
lightgbm	Light Gradient Boosting Machine	0.7225	0.7811	0.5506	0.6231	0.5779	0.3739	0.3804	0.1640
ada	Ada Boost Classifier	0.7336	0.7854	0.5398	0.6396	0.5749	0.3865	0.3961	0.1810
knn	K Neighbors Classifier	0.7116	0.7582	0.5205	0.6137	0.5531	0.3448	0.3539	0.1760
dt	Decision Tree Classifier	0.6944	0.6529	0.5173	0.5689	0.5379	0.3118	0.3144	0.1630
svm	SVM - Linear Kernel	0.5694	0.0000	0.6307	0.4608	0.4685	0.1562	0.1922	0.1540
dummy	Dummy Classifier	0.6518	0.5000	0.0000	0.0000	0.0000	0.0000	0.0000	0.1660

	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC
Fold							
0	0.7963	0.8797	0.8421	0.6667	0.7442	0.5787	0.5896
1	0.7778	0.8451	0.5263	0.7692	0.6250	0.4749	0.4921
2	0.7963	0.8902	0.5789	0.7857	0.6667	0.5248	0.5375
3	0.7593	0.8090	0.6842	0.6500	0.6667	0.4785	0.4788
4	0.8148	0.8000	0.6842	0.7647	0.7222	0.5840	0.5860
5	0.7778	0.8150	0.5263	0.7692	0.6250	0.4749	0.4921
6	0.7593	0.7835	0.4211	0.8000	0.5517	0.4081	0.4474
7	0.8302	0.8286	0.6667	0.8000	0.7273	0.6055	0.6108
8	0.7547	0.7556	0.4444	0.7273	0.5517	0.3961	0.4189
9	0.7358	0.8683	0.4444	0.6667	0.5333	0.3592	0.3736
Mean	0.7802	0.8275	0.5819	0.7399	0.6414	0.4885	0.5027
Std	0.0278	0.0412	0.1287	0.0553	0.0735	0.0803	0.0741

optimization loop finished successfully. Best threshold: 0.3203125 with F1=0.6691

CustomProbabilityThresholdClassifier Confusion Matrix

